14 Working with RSocket

This chapter covers

- Reactive network communication with RSocket
- Working with each of RSocket's four communication models
- Transporting RSocket over WebSocket

There was a time, before telephones and modern electronics, when the best way to communicate with friends and family that live far away involved writing a letter and dropping it in the mail. It wasn't a quick form of communication, taking several days or even weeks before you'd receive a response, but it was effective and truly the only option available.

Thanks to Alexander Graham Bell, the telephone offered a new way to talk with distant friends and family, giving near-real-time, synchronous communication The telephone has evolved quite a bit since Mr. Bell's first invention, but it's still a popular means of keeping in touch, making letter-writing nearly a lost art.

When it comes to communication between applications, the request-response model offered by HTTP and REST services is quite common, but it has limitations. Much like letter-writing, request-response involves sending a message and then waiting for a response. It doesn't easily allow for asynchronous communication in which a server might respond with a stream of responses or allow for an open bidirectional channel on which a client and server can repeatedly send data to each other.

In this chapter, we're going to look at RSocket, a relatively new protocol for interapplication communication that allows for more than simple request-response communication. And because it's reactive in nature, it can be far more efficient than blocking HTTP requests.

Along the way, we'll see how to develop RSocket communication in Spring. But first, let's take a high-level look at RSocket to see what makes it different from HTTP-based communication.

14.1 Introducing RSocket

RSocket (https://rsocket.io/) is a binary application protocol that is asynchronous and based on Reactive Streams. Put another way, RSocket offers asynchronous communication between applications that supports a reactive model consistent with reactive types like `Flux` and `Mono` that we learned about in chapter 12.

As an alternative to HTTP-based communication, it is more flexible, providing four distinct communication models: request-response, request-stream, fire-and-forget, and channel.

Request-response is the most familiar communication model from RSocket, mimicking how typical HTTP communication works. In the request-response model, a client issues a single request to the server, and the server responds with a single response. This is illustrated in figure 14.1, using Reactor's `Mono` type to define the request and response.

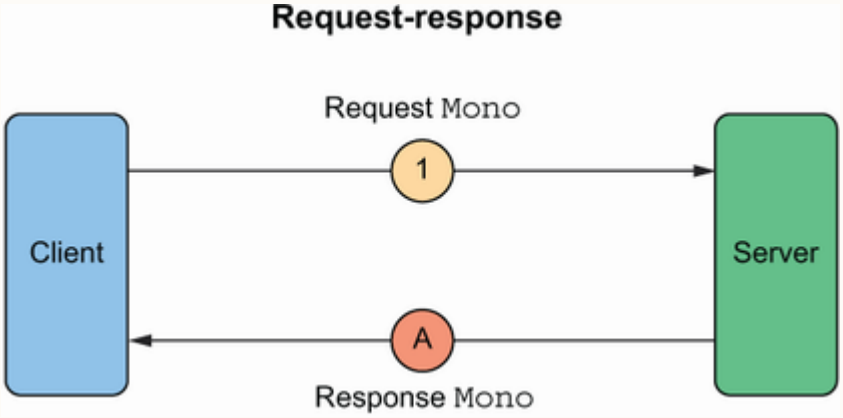


Figure 14.1 RSocket's request-response communication model

Although the request-response model may appear to be equivalent to the communication model offered by HTTP, it's important to understand that RSocket is fundamentally nonblocking and based on reactive types. Although the client will still wait for a reply from the server, under the covers everything is nonblocking and reactive, making more efficient use of threads.

The *request-stream* communication model is similar to request-response, except that after the client has sent a single request to the server, the server responds with a stream of zero-to-many values in a stream. Figure 14.2 illustrates the request-stream model using `Mono` for the request and `Flux` for the response.

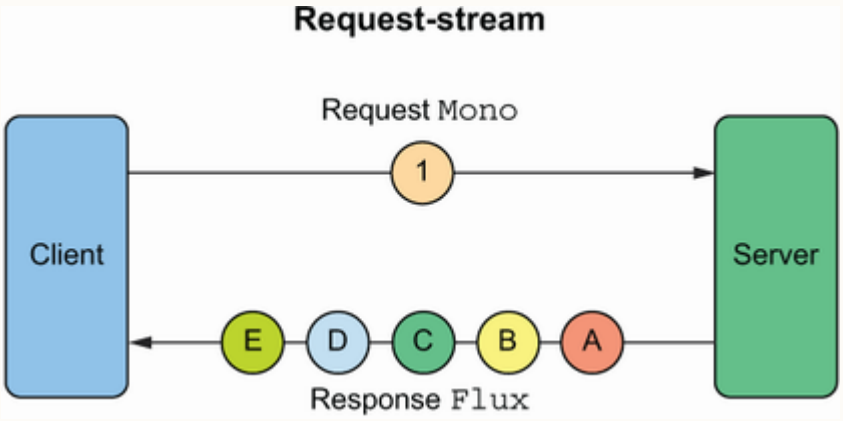


Figure 14.2 RSocket's request-stream communication model

In some cases, the client may need to send data to the server but doesn't need a response. RSocket provides the *fire-and-forget* model for those situations, as illustrated in figure 14.3.

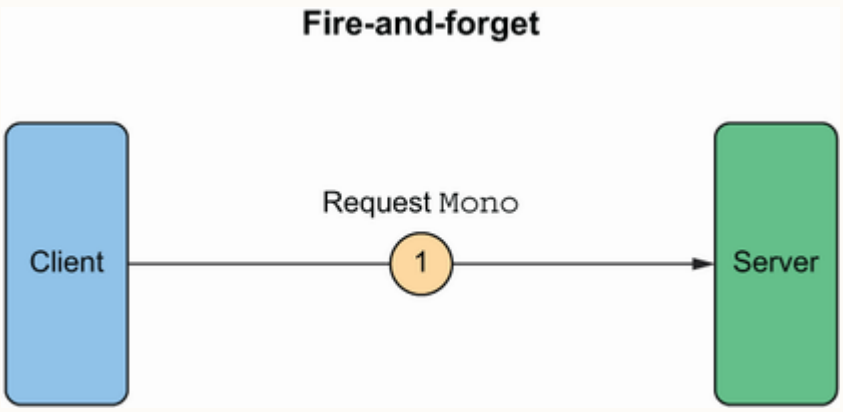


Figure 14.3 RSocket's fire-and-forget communication model

In the fire-and-forget model, a client sends a request to the server, but the server doesn't send a response back.

Finally, the most flexible of RSocket's communication models is the *channel* model. In the channel model, the client opens a bidirectional channel with the server, and each can send data to the other at any time. Figure 14.4 illustrates the channel communication style.

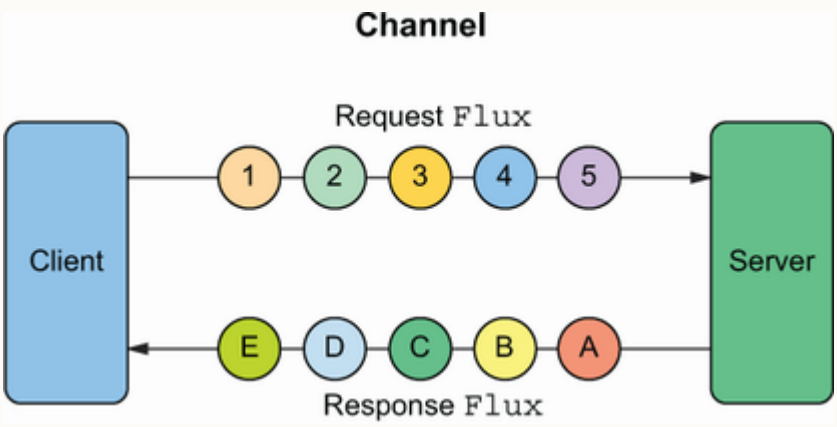


Figure 14.4 RSocket's channel communication model

RSocket is supported on a variety of languages and platforms, including Java, JavaScript, Kotlin, .NET, Go, and C++.[1] Recent versions of Spring offer first-class support for RSocket, making it easy to create servers and clients using familiar Spring idioms.

Let's dive in and see how to create RSocket servers and clients that work with each of the four communication models.

14.2 Creating a simple RSocket server and client

Spring offers incredible support for messaging with RSocket, including all four communication models. To get started with RSocket, you'll need to add the Spring Boot RSocket starter to your project's build. In a Maven POM file, the RSocket starter dependency looks like this the following.

Listing 14.1 Spring Boot's RSocket starter dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>
```

This same dependency is needed for both the server and client applications involved in RSocket communication.

NOTE When choosing dependencies from the Spring Initializr, you might see a similarly named WebSocket dependency. Although RSocket and WebSocket have similar names and although you can use WebSocket as a transport for RSocket (and we'll cover that later in this chapter), you do not need to select the WebSocket dependency when working with RSocket.

Next, you'll need to decide which communication model is best for your application. There's no clear answer that fits every situation, so you'll want to weigh the choice against the desired communication behavior of your application. However, as you'll see in the next several examples, the development model isn't much different for each of the communication models, so it'll be easy to switch if you choose wrong.

Let's see how to create an RSocket server and client in Spring using each of the communication models. Because each of RSocket's communication models is different and is best suited for specific use-case scenarios, we'll set the Taco Cloud application aside for now and see how to apply RSocket on different problem domains. We'll start by seeing how to apply the request-response communication model.

14.2.1 Working with request-response

Creating an RSocket server in Spring is as simple as creating a controller class, much the same as you would for a web application or REST service. The following controller is an example of an RSocket service that handles greetings from the client and responds with another greeting.

Listing 14.2 A simple RSocket request-response server

```
package rsocket;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;

@Controller
@Slf4j
public class GreetingController {

  @MessageMapping("greeting")
  public Mono<String> handleGreeting(Mono<String> greetingMono) {
    return greetingMono
      .doOnNext(greeting ->
        log.info("Received a greeting: {}", greeting))
      .map(greeting -> "Hello back to you!");
  }

}
```

As you can see, the key difference between a web controller and an RSocket controller is that instead of handling HTTP requests for a given path (using `@GetMapping` or `@PostMapping`), an RSocket controller handles incoming messages on a given route with the `@MessageMapping` annotation. In this example, the `handleGreeting()` method is invoked when a request is sent from the client to the route named `"greeting"`.

The `handleGreeting()` method receives the message payload from the client in a `Mono<String>` parameter. In this case, the greeting is simple enough that a `String` is sufficient, but the incoming payload could be a more complex type, if needed. Upon receiving the `Mono<String>`, the method simply logs the fact that it received the greeting and then uses the `map()` function on the `Mono` to create a new `Mono<String>` to carry the response that is returned to the client.

Although RSocket controllers aren't handling HTTP requests for a path, the route name can be made to have a pathlike appearance, including variable placeholders that can be passed into the handler method. For example, consider the following twist on the `handleGreeting()` method:

```
@MessageMapping("greeting/{name}")
public Mono<String> handleGreeting(
      @DestinationVariable("name") String name,
      Mono<String> greetingMono) {

   return greetingMono
      .doOnNext(greeting ->
         log.info("Received a greeting from {} : {}", name, greeting))
      .map(greeting -> "Hello to you, too, " + name);
}
```

In this case, the route specified in `@MessageMapping` contains a placeholder variable named `"name"`. It is denoted by curly braces, the same way as path variables in a Spring MVC controller. Likewise, the method accepts a `String` parameter annotated with `@DestinationVariable` that references the placeholder variable. Just like Spring MVC's `@PathVariable` annotation, `@DestinationVariable` is used to extract the value specified in the route's placeholder and pass it into the handler method. Once inside this new version of `handleGreeting()`, the name specified in the route will be used to return a more personalized greeting to the client.

There's one more thing you must remember to do when creating an RSocket server: specify the port to listen on. By default, RSocket services are TCP-based and are their

own server listening on a specific port. The `spring.rsocket.server.port` configuration property sets the port for the RSocket server, as shown here:

```
spring:
  rsocket:
    server:
      port: 7000
```

The `spring.rsocket.server.port` property serves two purposes: enabling a server and specifying which port the server should listen on. If it is not set, then Spring will assume that your application will be acting as a client only, and no server port will be listening. In this case, we're starting a server, so setting the `spring.rsocket .server.port` property as shown in the previous code will start a server listening on port 7000.

Now let's turn our attention to the RSocket client. In Spring, RSocket clients are implemented using an `RSocketRequester`. Spring Boot autoconfiguration for RSocket will automatically create a bean of type `RSocketRequester.Builder` in the Spring application context. You can inject that builder bean into any other bean you need to create an instance of `RSocketRequester`.

For example, here's the start of an `ApplicationRunner` bean that is injected with an `RSocketRequester.Builder`:

```
package rsocket;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.rsocket.RSocketRequester;

@Configuration
@Slf4j
public class RSocketClientConfiguration {

  @Bean
  public ApplicationRunner sender(RSocketRequester.Builder requesterBuilder) {
    return args -> {
      RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

      // ... send messages with RSocketRequester ...

    };
  }
```

```
}
```

In this case, the builder is used to create an `RSocketRequester` that listens on `localhost`, port 7000. The resulting `RSocketRequester` can then be used to send messages to the server.

In a request-response model, the request will need to (at least) specify the route and the data payload. As you'll recall, our server's controller is handling requests for the route named `"greeting"` and expects a `String` input. It also returns a `String` output. The following complete listing of client code shows how to send a greeting to the server and handle the response.

Listing 14.3 Sending a request from a client

```
RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

// ... send messages with RSocketRequester ...
tcp
  .route("greeting")
  .data("Hello RSocket!")
  .retrieveMono(String.class)
  .subscribe(response -> log.info("Got a response: {}", response));
```

This sends a greeting of `"Hello RSocket!"` to the server on the `"greeting"` route. Notice that it also expects a `Mono<String>` in return, as specified in the call to `retrieveMono()`. The `subscribe()` method subscribes to the returned `Mono` and handles its payload by logging the value.

Now let's say you want to send a greeting to the other route that accepts a variable value in its route. The client-side code works pretty much the same, except that you include the variable placeholder in the value given to `route()` along with the value it should contain as follows:

```
String who = "Craig";
tcp
  .route("greeting/{name}", who)
  .data("Hello RSocket!")
  .retrieveMono(String.class)
  .subscribe(response -> log.info("Got a response: {}", response));
```

Here, the message will be sent to the route named `"greeting/Craig"`, which will be handled by the controller handler method whose `@MessageMapping` specified the route `"greeting/{name}"`. Although you could also hardcode the name in the route or use `String` concatenation to create the route name, using a placeholder in the client makes it really easy to drop in a value without the messiness of `String` concatenation.

Although the request-response model is probably the easiest of RSocket's communication models to wrap your head around, it's just the beginning. Let's see how to handle requests that could potentially return several responses with the request-stream model.

14.2.2 Handling request-stream messaging

Not all interactions feature a single request and a single response. In a stock quote scenario, for example, it may be useful to request a stream of stock quotes for a given stock symbol. In a request-response model, the client would need to repeatedly poll for the current stock price. But in a request-stream model, the client need ask for the stock price only once and then subscribe to a stream of periodic updates.

To illustrate the request-stream model, let's implement the server and client for the stock quote scenario. First, we'll need to define an object that can carry the stock quote information. The `StockQuote` class in the next listing will serve this purpose.

Listing 14.4 A model class representing a stock quote

```
package rsocket;
import java.math.BigDecimal;
import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class StockQuote {

   private String symbol;
   private BigDecimal price;
   private Instant timestamp;

}
```

As you can see, a `StockQuote` carries the stock symbol, the price, and a timestamp that the price was valid. For brevity's sake, we're using Lombok to help with constructors and accessor methods.

Now let's write a controller to handle requests for stock quotes. You'll find that the `StockQuoteController` in the next snippet is quite similar to the `GreetingController` from the previous section.

Listing 14.5 An RSocket controller to stream stock quotes

```
package rsocket;
import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;

import org.springframework.messaging.handler.annotation.DestinationVariable;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import reactor.core.publisher.Flux;

@Controller
public class StockQuoteController {

   @MessageMapping("stock/{symbol}")
   public Flux<StockQuote> getStockPrice(
         @DestinationVariable("symbol") String symbol) {
      return Flux
         .interval(Duration.ofSeconds(1))
         .map(i -> {
            BigDecimal price = BigDecimal.valueOf(Math.random() * 10);
            return new StockQuote(symbol, price, Instant.now());
         });
   }
}
```

Here, the `getStockPrice()` method handles incoming requests on the `"stock/{symbol}"` route, accepting the stock symbol from the route with the `@DestinationVariable` annotation. For simplicity's sake, rather than look up actual stock prices, the price is calculated as a random value (which may or may not accurately model the volatility of some actual stocks).

What's most notable about `getStockPrice()`, however, is that it returns a `Flux<StockQuote>` instead of a `Mono<StockQuote>`. This is a clue to Spring that this handler method supports the request-stream model. Internally, the `Flux` is created initially as an interval that fires every one second, but that `Flux` is mapped to another `Flux` that produces the random `StockQuote`. Put simply, a single request handled by the `getStockPrice()` method returns multiple values, once every second.

A client of a request-stream service is similar to one for a request-response service. The only key difference is that instead of calling `retrieveMono()` on the requester, it should call `retreiveFlux()`. The client of the stock quote service might look like this:

```
String stockSymbol = "XYZ";

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
   .route("stock/{symbol}", stockSymbol)
   .retrieveFlux(StockQuote.class)
   .doOnNext(stockQuote ->
      log.info(
            "Price of {} : {} (at {})",
            stockQuote.getSymbol(),
            stockQuote.getPrice(),
            stockQuote.getTimestamp())
   )
   .subscribe();
```

At this point, we've seen how to create RSocket servers and clients that handle single and multiple responses. But what if the server doesn't have a response to send or the client doesn't need a response? Let's see how to deal with the fire-and-forget communication model.

4.2.3 Sending fire-and-forget messages

Imagine that you're on a starship that has just come under attack from an enemy vessel. You sound a ship-wide "red alert" so that all hands are in battle mode. You don't need to wait for a response from the ship's computers affirming the alert status, nor do you have time to wait for and read any kind of response in this situation. You set the alert and then move on to more critical matters.

This is an example of fire-and-forget. Although you may not forget that you're at red alert, given the circumstances, it's more important that you deal with the battle crisis

than it is for you to handle a response from setting the alert.

To simulate this scenario, we'll create an RSocket server that handles alert statuses but doesn't return anything. First, we'll need to define a class that defines the request payload, such as the `Alert` class in the following code listing.

Listing 14.6 A model class representing an alert

```
package rsocket;

import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class Alert {

   private Level level;
   private String orderedBy;
   private Instant orderedAt;

   public enum Level {
      YELLOW, ORANGE, RED, BLACK
   }
}
```

The `Alert` object captures the alert level, who ordered the alert, and a timestamp for when the alert was ordered (defined as an `Instant`). Again, we're using Lombok for constructors and accessor methods in the interest of keeping the listing short.

On the server side, the `AlertController` in the following listing will handle `Alert` messages.

Listing 14.7 An RSocket controller to handle alert updates

```
package rsocket;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;
```

```
@Controller
@Slf4j
public class AlertController {

   @MessageMapping("alert")
   public Mono<Void> setAlert(Mono<Alert> alertMono) {
      return alertMono
         .doOnNext(alert ->
            log.info("{} alert ordered by {} at {}",
                  alert.getLevel(),
                  alert.getOrderedBy(),
                  alert.getOrderedAt())
         )
         .thenEmpty(Mono.empty());
   }

}
```

The `setAlert()` method handles `Alert` messages on the `"alert"` route. To keep things simple (albeit useless in an actual battle situation), it logs only the alerts. But what's important is that it returns a `Mono<Void>`, indicating that there is no response, and, therefore, this handler method supports the fire-and-forget model.

In the client, the code isn't much different from the request-response or request-stream models, as shown here:

```
RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
   .route("alert")
   .data(new Alert(
         Alert.Level.RED, "Craig", Instant.now()))
   .send()
   .subscribe();
log.info("Alert sent");
```

Notice, however, that instead of calling `retrieveMono()` or `retrieveFlux()`, the client merely calls `send()` with no expectation of a response.

Now let's take a look at how to handle the channel communication model in which both the server and the client send multiple messages to each other.

4.2.4 Sending messages bidirectionally

In all of the communication models we've seen thus far, the client sends a single request, and the server responds with zero, one, or many responses. In the request-stream model, the server was able to stream back multiple responses to the client, but the client was still limited to sending only a single request. But why should the server have all of the fun? Why can't the client send multiple requests?

That's where the channel communication model comes in handy. In the channel communication model, the client can stream multiple requests to the server, which may also stream back multiple responses in a bidirectional conversation between both sides. It's the most flexible of RSocket's communication models, although also the most complex.

To demonstrate how to work with RSocket channel communication in Spring, let's create a service that calculates gratuity on a bill, receiving a `Flux` of requests and responding with a `Flux` of responses. First, we'll need to define the model objects that represent the request and the response. The `GratuityIn` class, shown next, represents the request sent by the client and received by the server.

Listing 14.8 A model representing an inbound gratuity request

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratuityIn {

   private BigDecimal billTotal;
   private int percent;

}
```

`GratuityIn` carries two essential pieces of information required to calculate gratuity: the bill total and a percentage. The `GratuityOut` class shown in the next code snippet represents the response, echoing the values given in `GratuityIn`, along with a `gratuity` property containing the calculated gratuity amount.

Listing 14.9 A model representing an outbound gratuity response

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratuityOut {

   private BigDecimal billTotal;
   private int percent;
   private BigDecimal gratuity;

}
```

The `GratuityController` in the next code listing handles the gratuity request and looks a lot like the controllers we've written earlier in this chapter.

Listing 14.10 An RSocket controller that handles multiple messages on a channel

```
package rsocket;
import java.math.BigDecimal;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Flux;

@Controller
@Slf4j
public class GratuityController {

   @MessageMapping("gratuity")
   public Flux<GratuityOut> calculate(Flux<GratuityIn> gratuityInFlux) {
      return gratuityInFlux
         .doOnNext(in -> log.info("Calculating gratuity:  {}", in))
         .map(in -> {
            double percentAsDecimal = in.getPercent() / 100.0;
            BigDecimal gratuity = in.getBillTotal()
                  .multiply(BigDecimal.valueOf(percentAsDecimal));
```

```
            return new GratuityOut(
                in.getBillTotal(), in.getPercent(), gratuity);
        });
    }

}
```

There is, however, one significant difference between the previous example and the earlier ones: not only does this code return a `Flux`, but it also accepts a `Flux` as input. As with the request-stream model, the `Flux` returned enables the controller to stream multiple values to the client. But the `Flux` parameter is what differentiates the channel model from the request-stream model. The `Flux` parameter coming in allows the controller to handle a stream of requests from the client coming into the handler method.

The client side of the channel model differs from the client of the request-stream model only in that it sends a `Flux<GratuityIn>` to the server instead of a `Mono<GratuityIn>`, as shown here.

Listing 14.11 A client that sends and receives multiple messages over an open channel

```
RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

Flux<GratuityIn> gratuityInFlux =
      Flux.fromArray(new GratuityIn[] {
            new GratuityIn(BigDecimal.valueOf(35.50), 18),
            new GratuityIn(BigDecimal.valueOf(10.00), 15),
            new GratuityIn(BigDecimal.valueOf(23.25), 20),
            new GratuityIn(BigDecimal.valueOf(52.75), 18),
            new GratuityIn(BigDecimal.valueOf(80.00), 15)
      })
      .delayElements(Duration.ofSeconds(1));

      tcp
         .route("gratuity")
         .data(gratuityInFlux)
         .retrieveFlux(GratuityOut.class)
         .subscribe(out ->
            log.info(out.getPercent() + "% gratuity on "
                  + out.getBillTotal() + " is "
                  + out.getGratuity()));
```

In this case, the `Flux<GratuityIn>` is created statically using the `fromArray()` method, but it could be a `Flux` created from any source of data, perhaps retrieved from a reactive data repository.

You may have observed a pattern in how the reactive types accepted and returned by the server controller's handler methods determine the RSocket communication model supported. Table 14.1 summarizes the relationship between the server's input/ output types and the RSocket communication models.

Table 14.1 The supported RSocket model is determined by the handler method's parameter and return types.

RSocket model	Handler parameter	Handler returns
Request-response	Mono	Mono
Request-stream	Mono	Flux
Fire-and-forget	Mono	Mono<Void>
Channel	Flux	Flux

You may wonder whether it's possible for a server to accept a `Flux` and return a `Mono`. In short, that's not an option. Although you may imagine handling multiple requests on an incoming `Flux` and responding with a `Mono<Void>` in a weird mashup of the channel and fire-and-forget models, there is no RSocket model that maps to that scenario. Therefore, it's not supported.

14.3 Transporting RSocket over WebSocket

By default, RSocket communication takes place over a TCP socket. But in some cases, TCP isn't an option. Consider the following two situations:

- The client is written in JavaScript and is running in a user's web browser.
- The client must cross a gateway or firewall boundary to get to the server, and the firewall doesn't allow communication over arbitrary ports.

Moreover, WebSocket itself lacks any support for routing, requiring that routing details be defined at the application level. By layering RSocket over WebSocket, WebSocket will benefit from RSocket's built-in routing support.

In these situations, RSocket can be transported over WebSocket. WebSocket communication takes place over HTTP, which is the primary means of communication in all web browsers and is usually allowed through firewalls.

To switch from the TCP transport to the WebSocket transport, you need to make only a few minor changes in the server and client. To start, because WebSocket is carried over HTTP, you need to be sure that the server-side application supports handling HTTP requests. Put simply, you need to add the following WebFlux starter dependency to the project build (if it's not already there):

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

You also need to specify that you want to use the WebSocket transport in the server-side configuration by setting the `spring.rsocket.server.transport` property. Also, you need to set the HTTP path that the RSocket communication will take place on by setting `spring.rsocket.server.mapping-path`. The server's configuration will look like this in application.yml:

```
spring:
  rsocket:
    server:
      transport: websocket
      mapping-path: /rsocket
```

Unlike the TCP transport, which communicates over a specific port, the WebSocket transport works over a specific HTTP path. Thus, there is no need to set `spring.rsocket.server.port` as with RSocket over TCP.

That's all you'll need to do on the server side to enable WebSocket transport for RSocket. Everything else will work exactly the same as with TCP.

On the client side, only one small change is required. Rather than create a TCP-based requester, you want to create a WebSocket-based requester by calling the `websocket()` method on the `RSocketRequester.Builder` like so:

```
RSocketRequester requester = requesterBuilder.websocket(
            URI.create("ws://localhost:8080/rsocket"));
```

```
requester
    .route("greeting")
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: {}", response));
```

And that's all there is to transporting RSocket over WebSocket!

Summary

- RSocket is an asynchronous binary protocol that offers four communication models: request-response, request-stream, fire-and-forget, and channel.
- Spring supports RSocket on the server through controllers and handler methods annotated with `@MessageMapping`.
- The `RSocketRequester` enables client-side communication with RSocket.
- In both cases, Spring's RSocket support works through Reactor's `Flux` and `Mono` reactive types for fully reactive communication.
- RSocket communication takes place over TCP by default but can also be transported over WebSocket to deal with firewall constraints and browser clients.

---

[1] This is just a short list of languages that are listed on the RSocket website, but there may be community-led implementations of RSocket for other languages.

Previous chapter
< 13 Persisting data reactively

Next chapter
Part 4. Deployed Spring >